Make a single .cpp file for all your classes. Write your own main function to test them.

Q1. A `Quote` class models undiscounted books. A book has a unique identified ISBN (of string type), and a net price (double) to represent the selling price. Its constructor should have two parameters: isbn and sale price. Net price for a quote having n books is n * sale price of that book.

The corresponding private members of the class can be named in any manner as you like, but the following public getters must be there:

```
string isbn() const;

double net_price(size_t n) const; // should return net price of n books of that ISBN number.
```

Q2. Write a global function matching the following declaration to print Quote details:

```
double print_total(ostream& os, const Quote& item, size_t n); // returns the net price
```

While calling print_total, pass cout in the place of argument os. In your own format, the function should print the ISBN of item, and its net price given the parameter n (so the quote is about net undiscounted price of n books).

Q3. Now, create a class `Bulk_quote` to model discounted books, as a derived class of Quote class. Books are sold with discounts (corresponding to a bulk quote), when a minimum number of books are ordered in a quote. The applicable discount percentage is also a user parameter. So, the net price calculation is to be done as follows: when number of books in the bulk quote is >= minimum quantity, discount is applied. Else, it is n * sales price.

Consequently, create a corresponding constructor, which accepts ISBN, undiscounted sales price, minimum quantity for discount, and discount %. Use the initializer list to call base constructor. Keep attributes private and use public getters for printing. Minimum quantity and discount % rate do not need any getters, because they are internal info.

- Which attribute of Quote class is sensible to make protected?
- Do you need to make any member method of Bulk_Quote class virtual?
- Can you use the print_total function to print details of a bulk quote?

Q4. Create another inherited class `Disc_quote` from Quote, which gives discount only if the undiscounted price crosses a threshold (e.g. 20% off for the quotes above 5000 rupees etc.). Modify the net_price calculation logic. Have proper constructor and relevant getters for this class. Test whether print_total can also print Disc_quote object.

Q5. Define copy constructor for the Quote class. Then define the copy constructors for the derived classes. Note that, to copy the base-part of the derived object, you need to call the base constructor using the initializer list in the derived copy constructor.

Q6. Create one `Basket` class, which can hold 10 Quote* in a member array. Its constructor does not take any parameter, and it has the following public methods:

```
void add_item(const Quote&); // adds the quote in the array. Do not call it more than 10 times. If called, ignore it.
```

```
double total_receipt(ostream&) const; // should print the total net prices of all the quotes in the basket.
```

In your main, create one Basket object, and put different kinds of Quote objects, and call total receipt to verify whether the totaling is correct.

Cistagram

Upto now, you have made the feature of following one user from any logged in user account. This makes followers array for an user. Similarly, add one following array, which can hold upto 10 user pointers. Adding one post adds a corresponding feed (pointer to the post) within own account, and in all of its followers. Add one method “unfollow” to user class, which removes the links between the follower/following users.

Make one class Story, which inherits Post class (convert the struct to class, have proper access specifiers. Do not make members attributes public, have public getters), and has one int “duration” member attribute which holds time in seconds (it is simulation, not actual seconds), and one counter “elapsed time”. One user can post a Story, which remains in the feed array for the duration amount. When the story is viewed once by any of the followers, elapsed time increases by 1. Each time doing +1 in this way when elapsed time reaches duration, the story is removed from the feed. No follower is then able to see that Story again. Story delete should work the same as Post delete.